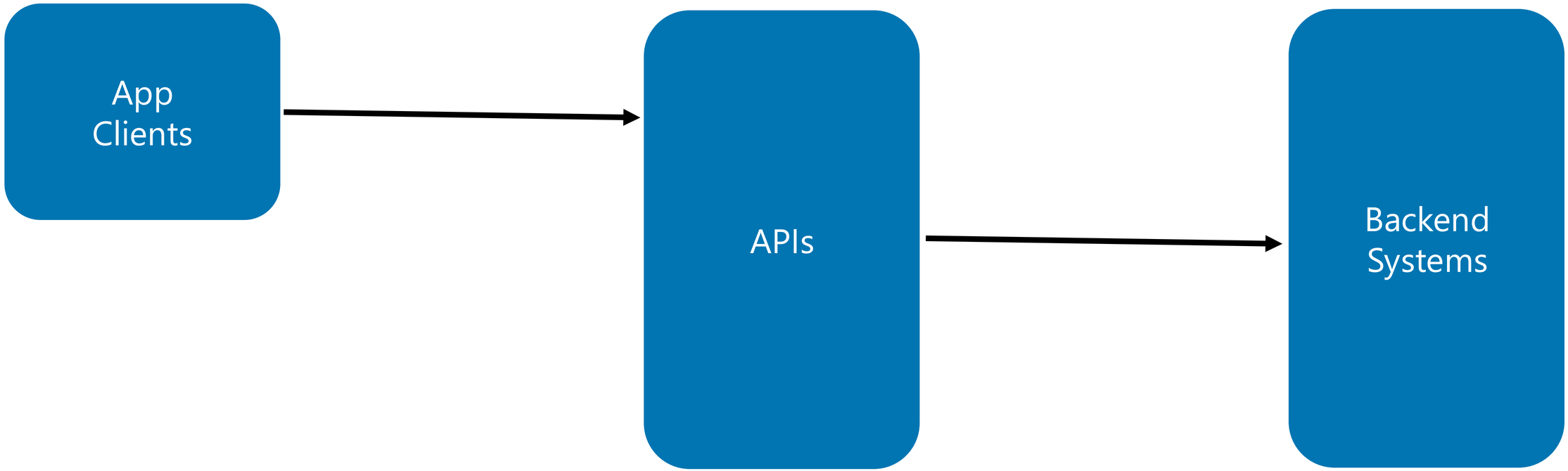
TRAILHEAD
TECHNOLOGY PARTNERS

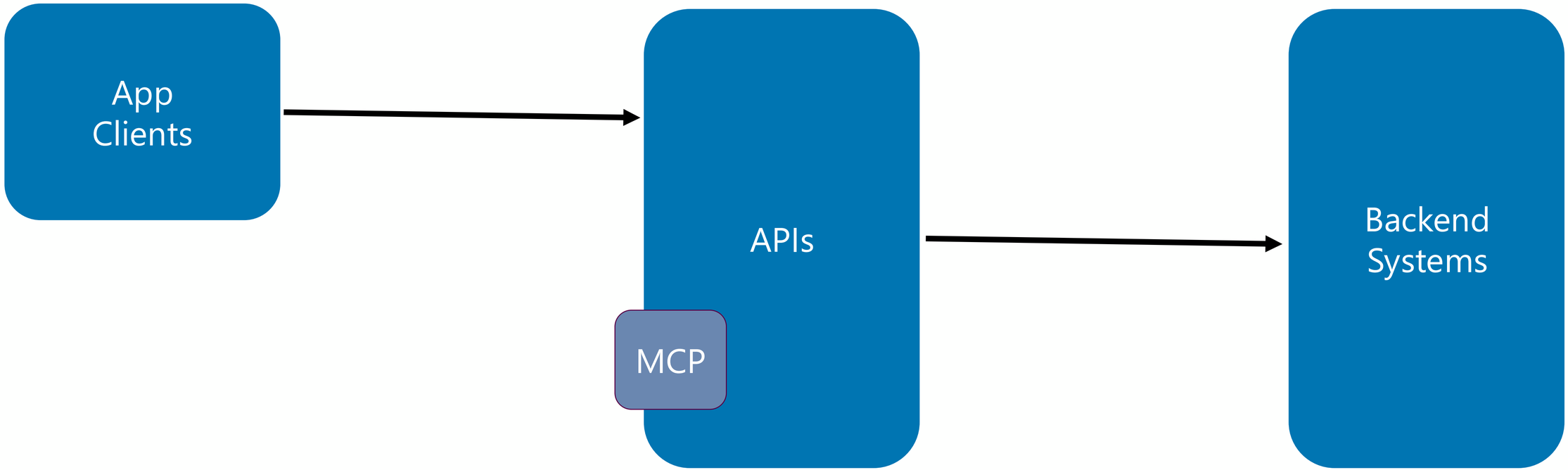
Old API, New Tricks

Add MCP to Existing .NET REST Endpoints

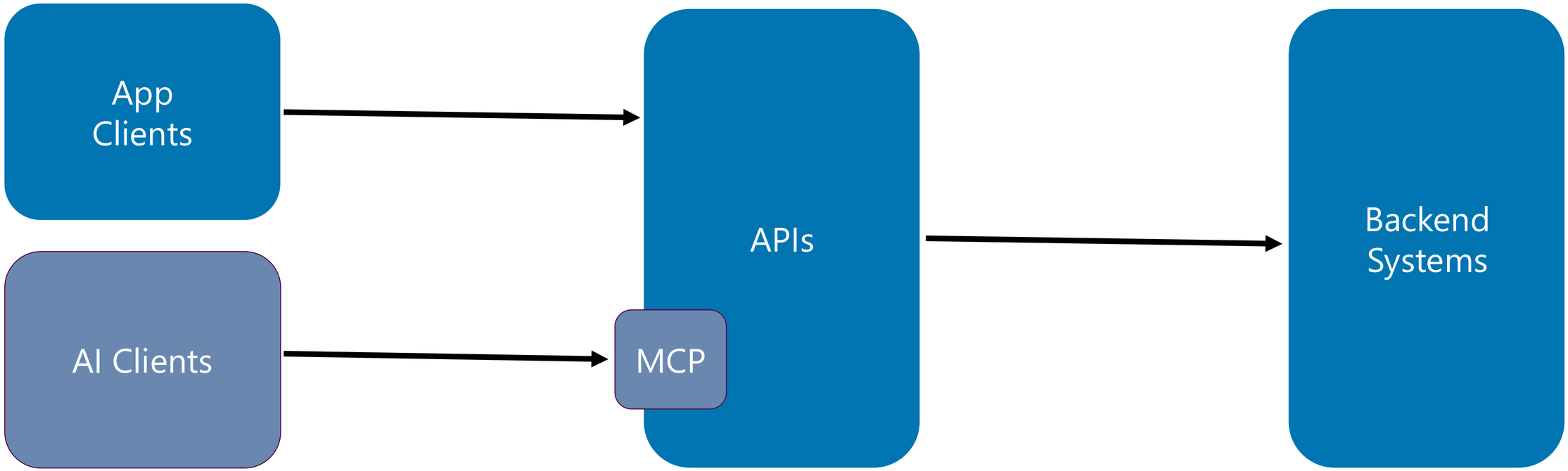
Jonathan “J.” Tower



TRAILHEAD
TECHNOLOGY PARTNERS



TRAILHEAD
TECHNOLOGY PARTNERS



TRAILHEAD
TECHNOLOGY PARTNERS

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

Document generation, assembling contracts from templates and data

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

Document generation, assembling contracts from templates and data

Procurement flow, creating POs, checking budgets, routing approvals

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

Document generation, assembling contracts from templates and data

Procurement flow, creating POs, checking budgets, routing approvals

Scheduling assistant, coordinating calendars, proposing times, sending invites

TRAILHEAD
TECHNOLOGY PARTNERS

Preview

01 What is MCP?
02 MCP vs REST
03 MCP C# SDK
04 Demo + Tool Design
05 Debugging

TRAILHEAD
TECHNOLOGY PARTNERS

Preview

01 What is MCP?
02 MCP vs REST
03 MCP C# SDK
04 Demo + Tool Design
05 Debugging

TRAILHEAD
TECHNOLOGY PARTNERS

Preview

01 What is MCP?
02 MCP vs REST
03 MCP C# SDK
04 Demo + Tool Design
05 Debugging

TRAILHEAD
TECHNOLOGY PARTNERS

Preview

01 What is MCP?
02 MCP vs REST
03 MCP C# SDK
04 Demo + Tool Design
05 Debugging

TRAILHEAD
TECHNOLOGY PARTNERS

Preview

01 What is MCP?
02 MCP vs REST
03 MCP C# SDK
04 Demo + Tool Design
05 Debugging

TRAILHEAD
TECHNOLOGY PARTNERS

Jonathan "J." Tower

TRAILHEAD TECHNOLOGY PARTNERS

Founding Partner

- 12x Microsoft MVP in .NET
- .NET Foundation Board
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- jtowermi
- Jonathan “J.” Tower

LET’S **TALK**

bit.ly/th-offer

github.com/trailheadtechnology/dotnet-mcp-rest

Introduction to MCP

TRAILHEAD
TECHNOLOGY PARTNERS

"MCP (Model Context Protocol) is an open-source standard introduced by Anthropic for connecting AI to external systems.

TRAILHEAD
TECHNOLOGY PARTNERS

"MCP (**Model Context Protocol**) is an open-source standard introduced by Anthropic for connecting AI to external systems.

TRAILHEAD
TECHNOLOGY PARTNERS

"MCP (**Model Context Protocol**) is an **open-source standard** introduced by Anthropic for connecting AI to external systems.

TRAILHEAD
TECHNOLOGY PARTNERS

"MCP (**Model Context Protocol**) is an **open-source standard** introduced by **Anthropic** for connecting AI to external systems.

TRAILHEAD
TECHNOLOGY PARTNERS

"MCP (**Model Context Protocol**) is an **open-source standard** introduced by **Anthropic** for **connecting AI** to external systems.

TRAILHEAD
TECHNOLOGY PARTNERS

"MCP (**Model Context Protocol**) is an **open-source standard** introduced by **Anthropic** for **connecting AI** to **external systems**.

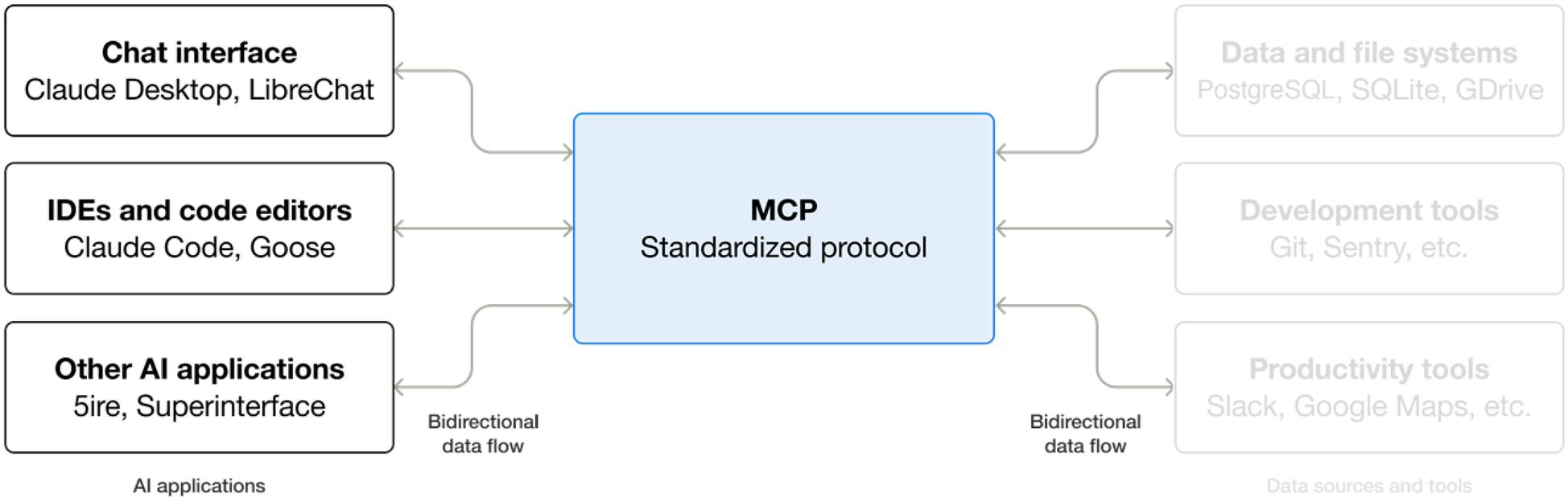
TRAILHEAD
TECHNOLOGY PARTNERS

Introduction to MCP

Source: https://modelcontextprotocol.io/docs/getting-started/intro

TRAILHEAD
TECHNOLOGY PARTNERS

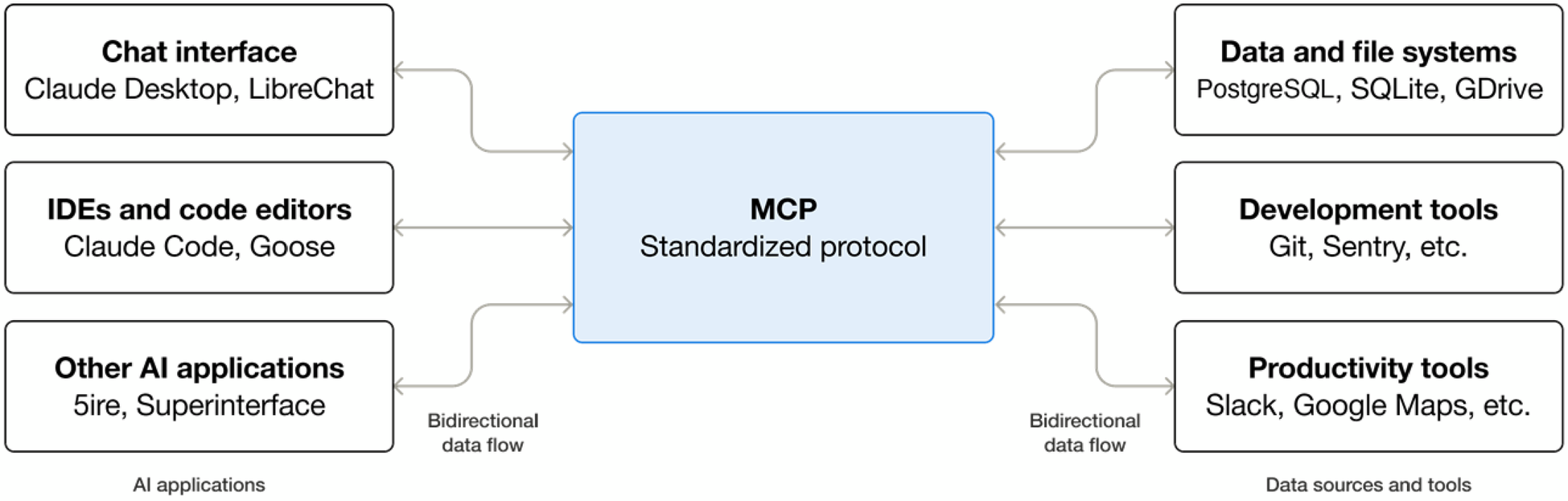
Introduction to MCP



Source: https://modelcontextprotocol.io/docs/getting-started/intro

TRAILHEAD
TECHNOLOGY PARTNERS

Introduction to MCP



Source: https://modelcontextprotocol.io/docs/getting-started/intro

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Transports

stdio

Local process over stdin/stout; no network, fast and simple.

Streamable HTTP

HTTP POST in, SSE stream out; remote-friendly, works with auth and proxies (formerly SSE)

Custom

Spec notes custom transports but not standardized

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Transports

stdio

Local process over stdin/out; no network, fast and simple.

Streamable HTTP

HTTP POST in, SSE stream out; remote-friendly, works with auth and proxies (formerly SSE)

Custom

Spec notes custom transports but not standardized

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Transports

stdio

Local process over stdin/out; no network, fast and simple.

Streamable HTTP

HTTP POST in, SSE stream out; remote-friendly, works with auth and proxies (formerly SSE)

Custom

Spec notes custom transports but not standardized

TRAILHEAD
TECHNOLOGY PARTNERS

REST vs MCP

TRAILHEAD
TECHNOLOGY PARTNERS

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

MCP C# SDK

TRAILHEAD
TECHNOLOGY PARTNERS

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

TRAILHEAD
TECHNOLOGY PARTNERS

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

TRAILHEAD
TECHNOLOGY PARTNERS

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

TRAILHEAD
TECHNOLOGY PARTNERS

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

TRAILHEAD
TECHNOLOGY PARTNERS

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

TRAILHEAD
TECHNOLOGY PARTNERS

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

TRAILHEAD
TECHNOLOGY PARTNERS

What's New in v1.x?

01

Enhanced Auth & Discovery

The SDK now supports a more flexible authorization model that separates the roles of the Authorization Server and Resource Server.

02

Incremental Scope Consent

Clients can now follow the Principle of Least Privilege by requesting only the minimum access needed for a specific task.

03

Tool Calling in Sampling

Servers can include tools within a sampling request, allow the LLM to gather more context before providing a final response.

04

Long-Running Reqs w/ Polling

Handling complex tasks over HTTP is improved through a standardized polling mechanism.

05

Experimental "Tasks" Support

The SDK introduces a new Tasks primitive for durable state tracking.

06

Richer Metadata & Icons

To improve user interfaces like GitHub Copilot Chat, resources and tools now support enhanced metadata.

TRAILHEAD
TECHNOLOGY PARTNERS

What's New in v1.x?

01

Enhanced Auth & Discovery

The SDK now supports a more flexible authorization model that separates the roles of the Authorization Server and Resource Server.

02

Incremental Scope Consent

Clients can now follow the Principle of Least Privilege by requesting only the minimum access needed for a specific task.

03

Tool Calling in Sampling

Servers can include tools within a sampling request, allow the LLM to gather more context before providing a final response.

04

Long-Running Reqs w/ Polling

Handling complex tasks over HTTP is improved through a standardized polling mechanism.

05

Experimental "Tasks" Support

The SDK introduces a new Tasks primitive for durable state tracking.

06

Richer Metadata & Icons

To improve user interfaces like GitHub Copilot Chat, resources and tools now support enhanced metadata.

TRAILHEAD
TECHNOLOGY PARTNERS

What's New in v1.x?

01

Enhanced Auth & Discovery

The SDK now supports a more flexible authorization model that separates the roles of the Authorization Server and Resource Server.

02

Incremental Scope Consent

Clients can now follow the Principle of Least Privilege by requesting only the minimum access needed for a specific task.

03

Tool Calling in Sampling

Servers can include tools within a sampling request, allow the LLM to gather more context before providing a final response.

04

Long-Running Reqs w/ Polling

Handling complex tasks over HTTP is improved through a standardized polling mechanism.

05

Experimental "Tasks" Support

The SDK introduces a new Tasks primitive for durable state tracking.

06

Richer Metadata & Icons

To improve user interfaces like GitHub Copilot Chat, resources and tools now support enhanced metadata.

TRAILHEAD
TECHNOLOGY PARTNERS

What's New in v1.x?

01

Enhanced Auth & Discovery

The SDK now supports a more flexible authorization model that separates the roles of the Authorization Server and Resource Server.

02

Incremental Scope Consent

Clients can now follow the Principle of Least Privilege by requesting only the minimum access needed for a specific task.

03

Tool Calling in Sampling

Servers can include tools within a sampling request, allow the LLM to gather more context before providing a final response.

04

Long-Running Reqs w/ Polling

Handling complex tasks over HTTP is improved through a standardized polling mechanism.

05

Experimental "Tasks" Support

The SDK introduces a new Tasks primitive for durable state tracking.

06

Richer Metadata & Icons

To improve user interfaces like GitHub Copilot Chat, resources and tools now support enhanced metadata.

TRAILHEAD
TECHNOLOGY PARTNERS

What's New in v1.x?

01

Enhanced Auth & Discovery

The SDK now supports a more flexible authorization model that separates the roles of the Authorization Server and Resource Server.

02

Incremental Scope Consent

Clients can now follow the Principle of Least Privilege by requesting only the minimum access needed for a specific task.

03

Tool Calling in Sampling

Servers can include tools within a sampling request, allow the LLM to gather more context before providing a final response.

04

Long-Running Reqs w/ Polling

Handling complex tasks over HTTP is improved through a standardized polling mechanism.

05

Experimental "Tasks" Support

The SDK introduces a new Tasks primitive for durable state tracking.

06

Richer Metadata & Icons

To improve user interfaces like GitHub Copilot Chat, resources and tools now support enhanced metadata.

TRAILHEAD
TECHNOLOGY PARTNERS

What's New in v1.x?

01

Enhanced Auth & Discovery

The SDK now supports a more flexible authorization model that separates the roles of the Authorization Server and Resource Server.

02

Incremental Scope Consent

Clients can now follow the Principle of Least Privilege by requesting only the minimum access needed for a specific task.

03

Tool Calling in Sampling

Servers can include tools within a sampling request, allow the LLM to gather more context before providing a final response.

04

Long-Running Reqs w/ Polling

Handling complex tasks over HTTP is improved through a standardized polling mechanism.

05

Experimental "Tasks" Support

The SDK introduces a new Tasks primitive for durable state tracking.

06

Richer Metadata & Icons

To improve user interfaces like GitHub Copilot Chat, resources and tools now support enhanced metadata.

TRAILHEAD
TECHNOLOGY PARTNERS

DEMO

TRAILHEAD
TECHNOLOGY PARTNERS

Tool Design Quality

TRAILHEAD
TECHNOLOGY PARTNERS

Descriptions Are Your API Contract

```
GET  /api/todos
POST /api/todos
PATCH /api/todos/{id}/complete
```

```
[Description("Returns all todos. Pass
true for completed, false for
incomplete, or omit to get all.")]
```

TRAILHEAD
TECHNOLOGY PARTNERS

Anatomy of a Good Description

What it does

When to use it

Valid param values

Param units and format

Return value expectation

TRAILHEAD
TECHNOLOGY PARTNERS

Anatomy of a Good Description

What it does

When to use it

Valid param values

Param units and format

Return value expectation

TRAILHEAD
TECHNOLOGY PARTNERS

Anatomy of a Good Description

What it does

When to use it

Valid param values

Param units and format

Return value expectation

TRAILHEAD
TECHNOLOGY PARTNERS

Anatomy of a Good Description

What it does

When to use it

Valid param values

Param units and format

Return value expectation

TRAILHEAD
TECHNOLOGY PARTNERS

Anatomy of a Good Description

What it does

When to use it

Valid param values

Param units and format

Return value expectation

TRAILHEAD
TECHNOLOGY PARTNERS

Bad vs. Good Descriptions ❌

```
[Description("Gets todos")]
public string GetTodos(bool? complete)
```

TRAILHEAD
TECHNOLOGY PARTNERS

Bad vs. Good Descriptions ❌

```
[Description("Gets todos")]
public string GetTodos(bool? complete)
```

Too simple

TRAILHEAD
TECHNOLOGY PARTNERS

Bad vs. Good Descriptions ❌

```
[Description("Gets todos")]
public string GetTodos(bool? complete)
```

Too simple

Missing

TRAILHEAD
TECHNOLOGY PARTNERS

Bad vs. Good Descriptions ✅

```
[Description("Returns all todo items. Pass true to get only
completed items, false for only incomplete items, or omit
the parameter entirely to return all items regardless of
status.")]
public string GetTodos(
    [Description("Filter by completed status. true = completed
    only, false = incomplete only, omit = all items")]
    bool? complete = null)
```

TRAILHEAD
TECHNOLOGY PARTNERS

Debugging MCP

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Inspector

- ❌ curl
- ❌ OpenAPI / Swagger
- ❌ Postman

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Inspector

- ❌ curl
- ❌ OpenAPI / Swagger
- ❌ Postman

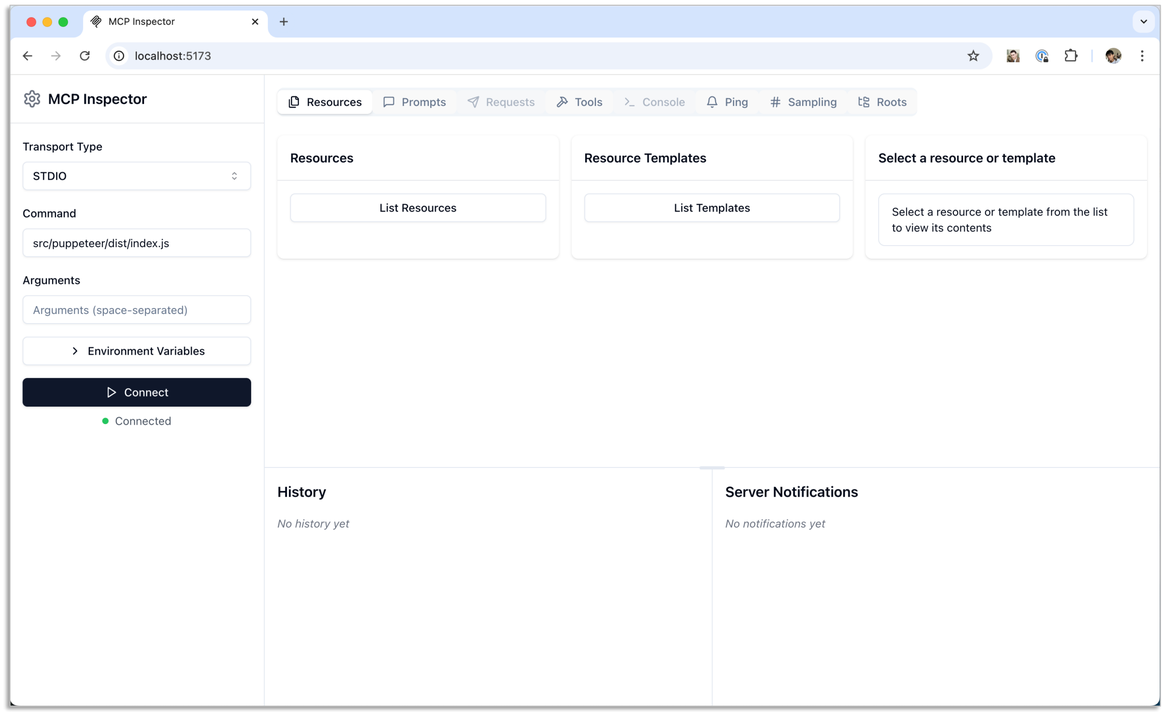
```
> npx @modelcontextprotocol/inspector <command>
```

TRAILHEAD
TECHNOLOGY PARTNERS

MCP Inspector

- ❌ curl
- ❌ OpenAPI / Swagger
- ❌ Postman

```
> npx @modelcontextprotocol/inspector <command>
```



TRAILHEAD
TECHNOLOGY PARTNERS

DEMO

TRAILHEAD
TECHNOLOGY PARTNERS

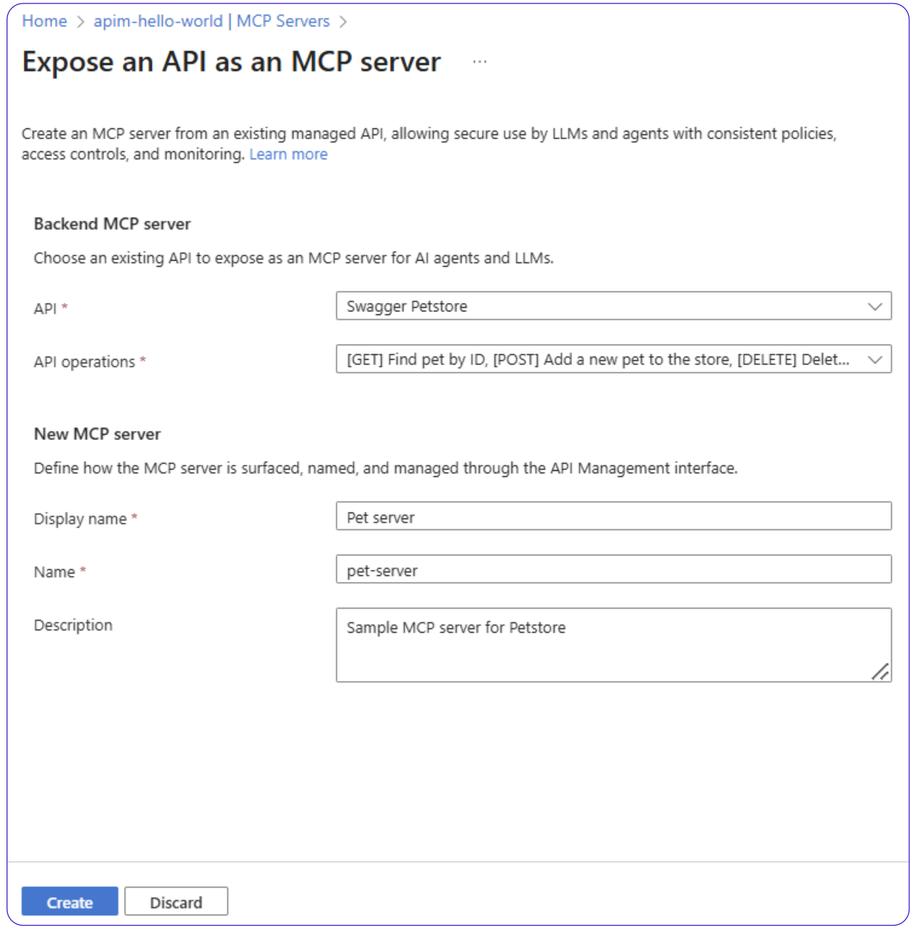
Other Options

TRAILHEAD
TECHNOLOGY PARTNERS

API Management

https://learn.microsoft.com/en-us/azure/api-management/export-rest-mcp-server

In API Management, you can expose a REST API managed in API Management as a remote Model Context Protocol (MCP) server using its built-in AI gateway



Azure Functions MCP Extension

https://learn.microsoft.com/en-us/azure/azure-functions/scenario-mcp-apps

```
[Function(nameof(GetWeather))]
public async Task<object> GetWeather(
    [McpToolTrigger(
         nameof(GetWeather),
        "Returns current weather for a location via Open-Meteo.")]
    [McpMetadata(ToolMetadata)]
    ToolInvocationContext context,

    [McpToolProperty(
        "location",
        "City name to check weather for (e.g., Seattle, New York, Miami)")]
    string location)
{
    ...
}
```

TRAILHEAD
TECHNOLOGY PARTNERS

Summing Up

1. MCP can be bolted on to existing REST APIs
2. While REST exposes resources, MCP exposes capabilities (not always 1:1)
3. Audience for REST is devs; for MCP it's AIs
4. LLMs can do multi-step orchestration
5. Tool descriptions are contracts, not just documentation
6. Debug with MCP Inspector

TRAILHEAD
TECHNOLOGY PARTNERS

Thanks! Questions?

Jonathan "J." Tower

- 12x Microsoft MVP in .NET
- .NET Foundation Board of Directors
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- jtowermi
- Jonathan “J.” Tower

LET’S
TALK

bit.ly/th-offer

github.com/trailheadtechnology/dotnet-mcp-rest